

Lab6 - Generative Models

StudentID : E94111253

1. Introduction

In this lab, I implemented a conditional Denoising Diffusion Probabilistic Model (DDPM) for multi-label image generation on the i-CLEVR dataset. Given a set of object conditions such as “red sphere”, “cyan cylinder”, or “yellow cube”, the model is required to generate a synthetic image containing the corresponding objects. Since each condition may contain multiple labels, the model needs to learn both object appearance and multi-label semantic consistency.

The generated images are evaluated using a pretrained ResNet18-based classifier provided by the instructor. The evaluator takes generated images and their ground-truth multi-hot labels as input and returns the classification accuracy. Therefore, the main objective of this lab is to generate images that are visually meaningful and semantically consistent with the given label conditions.

In this report, I describe my dataset preprocessing, conditional embedding method, U-Net-based DDPM architecture, noise schedule, training strategy, sampling method, and experimental results.

2. Implementation Details

2.1 Dataset and Multi-label Condition Encoding

The dataset contains synthetic i-CLEVR images with object labels. The object labels are defined in `objects.json`, where each color-shape combination corresponds to one of 24 object classes. For each training image, I read its label list from `train.json` and converted it into a 24-dimensional multi-hot vector. For example, if an image contains `["red cube", "green sphere", "brown cylinder"]`, the corresponding dimensions of these three labels are set to 1, while all other dimensions remain 0.

All images are resized to 64×64 and normalized using:

```
Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
```

This converts image values from `[0, 1]` to `[-1, 1]`. During image saving, I apply denormalization to convert generated images back to RGB PNG files. This follows the evaluator requirement that the input image should be normalized, while saved generated images should be denormalized.

2.2 Conditional DDPM Architecture

I designed a conditional U-Net as the denoising network of the DDPM. The model takes three inputs:

- x_t : noisy image at timestep t
- t : diffusion timestep
- c : 24-dimensional multi-hot condition vector

The U-Net consists of an encoder, a bottleneck, and a decoder. The encoder downsamples feature maps from 64×64 to 16×16 , and the decoder upsamples them back to 64×64 . Skip connections are used to preserve spatial information. To improve global reasoning for multi-object generation, I added a self-attention block at the 16×16 bottleneck.

For timestep information, I used sinusoidal time embedding followed by an MLP. For label conditioning, the 24-dimensional multi-hot vector is projected by a linear embedding network. The time embedding and condition embedding are combined as:

$$\text{emb} = \tanh(\text{time_embedding} + \text{condition_embedding})$$

The combined embedding is injected into each residual block, allowing the model to condition the denoising process on both timestep and object labels.

2.3 DDPM Objective and Noise Schedule

I used the standard DDPM epsilon prediction objective. During training, a random timestep t is sampled for each image, and Gaussian noise is added to the clean image according to:

$$x_t = \sqrt{\alpha_{\bar{t}}} * x_0 + \sqrt{1 - \alpha_{\bar{t}}} * \epsilon$$

The model is trained to predict the added noise ϵ . The loss function is mean squared error:

$$L = \text{MSE}(\epsilon_{\theta}(x_t, t, c), \epsilon)$$

I used a linear beta schedule with 1000 diffusion timesteps. The beta values increase linearly from $1e-4$ to 0.02 .

2.4 Classifier-free Guidance and EMA

To strengthen conditional generation, I used classifier-free guidance. During training, the condition vector is randomly dropped with probability 0.1 and replaced by a zero vector. Therefore, the model learns both conditional and unconditional denoising. During sampling, the conditional and unconditional predictions are combined using a guidance

scale.

I also used exponential moving average (EMA) to stabilize sampling. After each optimizer update, an EMA copy of the model parameters is updated. During sampling, I use the EMA weights instead of the raw model weights to generate more stable images.

2.5 Sampling and Testing

For testing, I used DDPM reverse sampling. Starting from Gaussian noise, the model iteratively denoises the image from timestep T to timestep 0. At each step, the model predicts the noise conditioned on the given multi-label vector.

For `test.json` and `new_test.json`, I generated 32 images for each file. The generated images were saved as PNG files following the order of the JSON conditions. I also generated the denoising process image using the label set ["red sphere", "cyan cylinder", "cyan cube"].

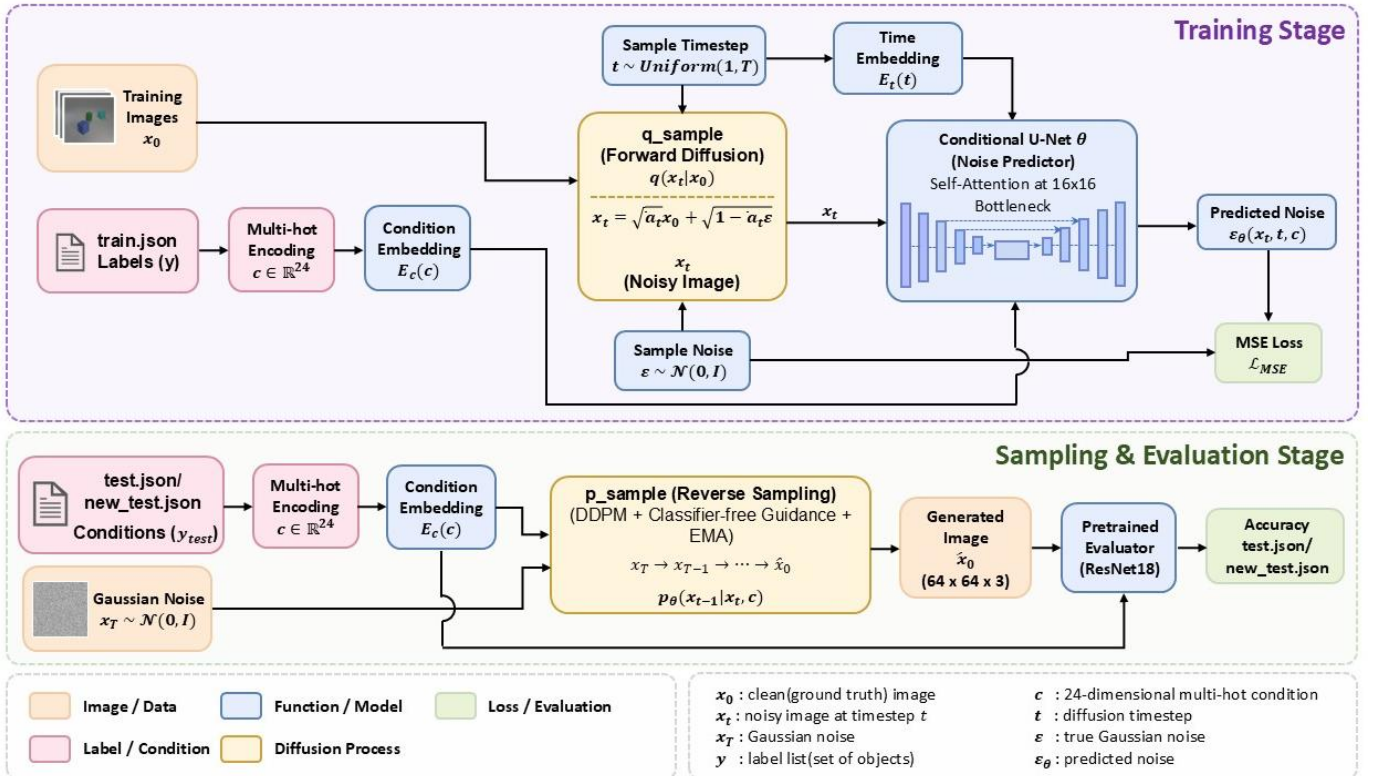


Fig. 1. Overall framework of the proposed conditional DDPM

Table 1. Model Configuration

Component	Design
Generative model	Conditional DDPM
Denoising network	Conditional U-Net
Input image size	$64 \times 64 \times 3$
Condition type	24-dimensional multi-hot vector
Time embedding	Sinusoidal time embedding + MLP
Condition embedding	Linear embedding + MLP
Noise prediction target	Epsilon prediction
Noise schedule	Linear beta schedule
Number of timesteps	1000
Sampling method	DDPM reverse sampling
Guidance method	Classifier-free guidance
EMA	Used for sampling
Attention	Self-attention at 16×16 bottleneck

Table 2. Training Settings

Hyperparameter	Value
Epochs	100 initial training + 100 fine-tuning
Batch size	64
Optimizer	AdamW
Initial learning rate	$2e-4$
Fine-tuning learning rate	$1e-4$
Weight decay	$1e-4$
Gradient clipping	1.0
Condition drop probability	0.1
EMA decay	0.999
Save interval	Every 10 epochs
Training data	i-CLEVR training images only

3. Results and Discussion

3.1 Synthetic Image Grids

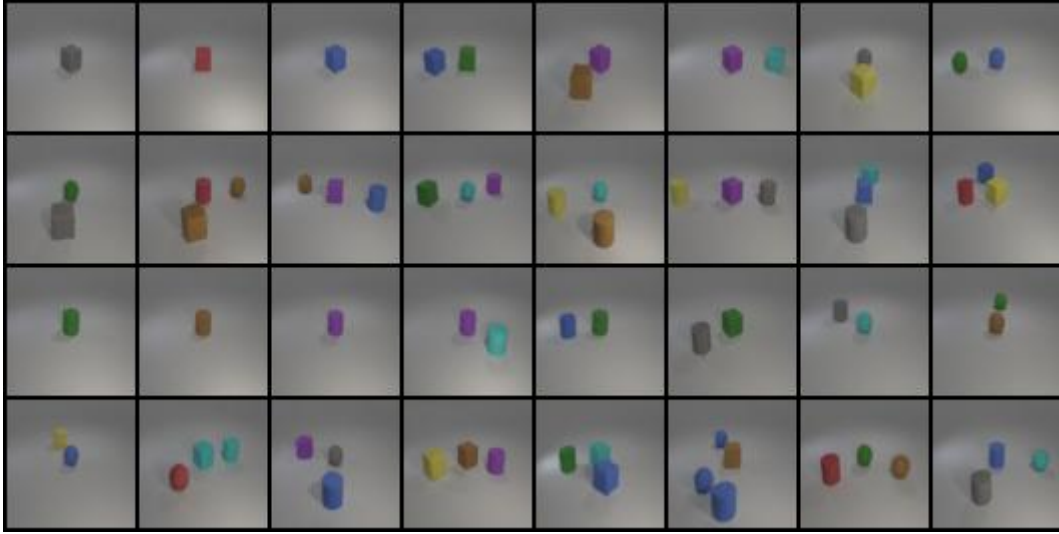


Fig. 2. Synthetic image grid generated from `test.json`.

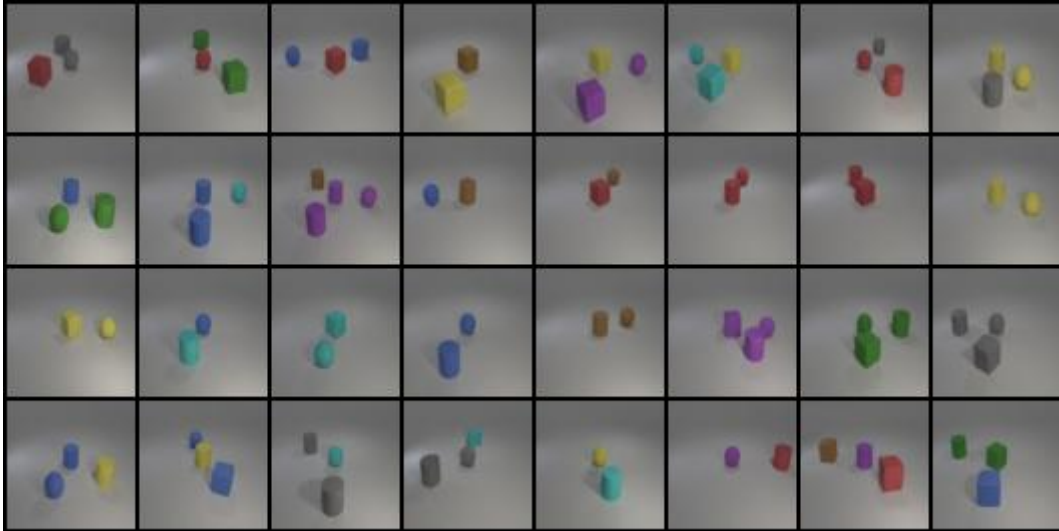


Fig. 3. Synthetic image grid generated from `new_test.json`

3.2 Denoising Process

The Figure 4. shows the reverse diffusion process from Gaussian noise to a generated image. The image gradually becomes clearer in later timesteps, where the model forms the target objects according to the given multi-label condition.



Fig. 4. Denoising process with condition ["red sphere", "cyan cylinder", "cyan cube"].

3.3 Effect of Classifier-free Guidance

The guidance scale comparison shows that classifier-free guidance is important for improving label consistency. As shown in Table 4, different guidance scales lead to different results on `test.json` and `new_test.json`, indicating that sampling performance is sensitive to the guidance value.

Table 4. Guidance Scale Comparison

Guidance Scale	<code>test.json</code> Accuracy	<code>new_test.json</code> Accuracy	Average
1.5	0.7778	0.9048	0.8413
2.0	0.7917	0.8214	0.8076
2.3	0.7222	0.9048	0.8135
2.5	0.7778	0.8571	0.8175
2.6	0.8056	0.8452	0.8254

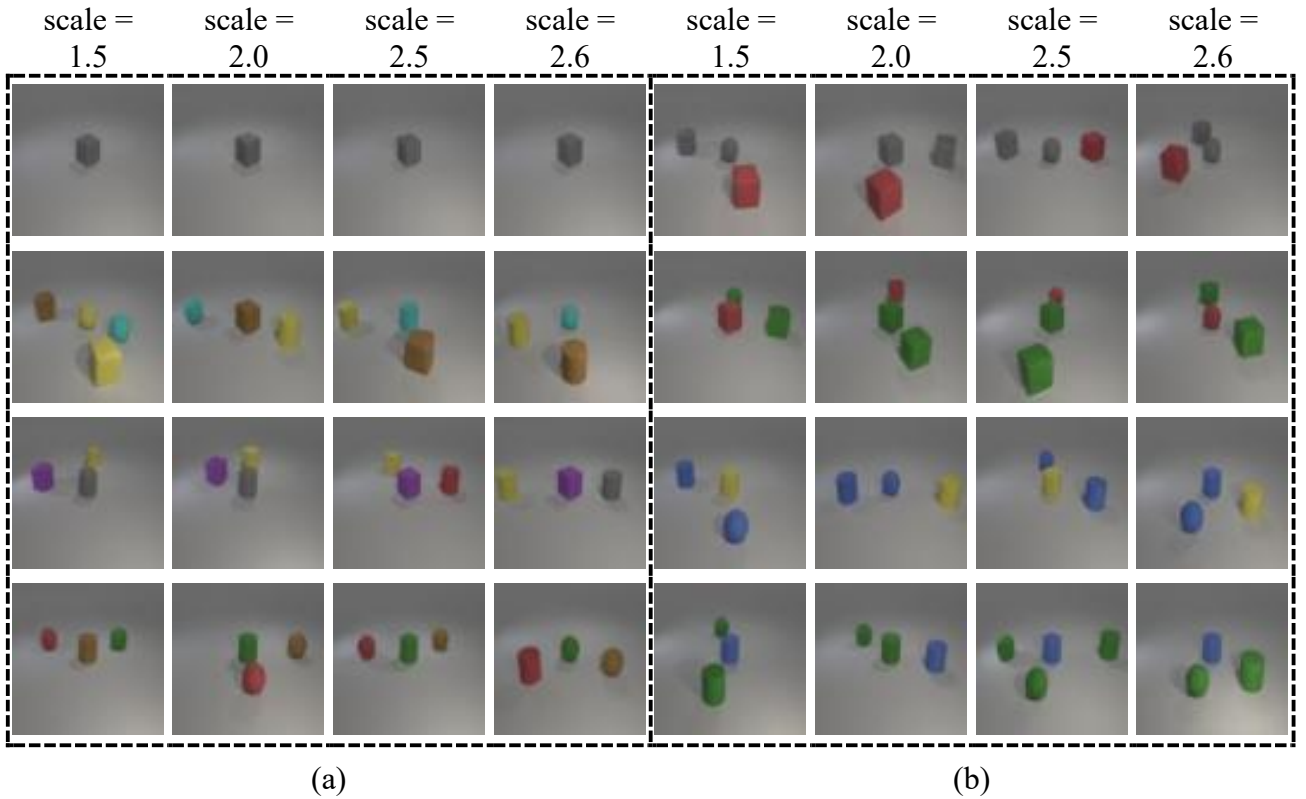


Fig. 5. Visual comparison of generated samples under different classifier-free guidance scales. (a) `test.json` samples. (b) `new_test.json` samples.

As shown in Table 4, different guidance scales affect `test.json` and `new_test.json` differently. Guidance scale 1.5 achieved the highest average accuracy, while guidance scale 2.6 achieved the best `test.json` accuracy. Since guidance scale 2.6 obtained 0.8056 on `test.json` and 0.8452 on `new_test.json`, both results exceeded the 0.8 threshold.

3.4 Effect of Attention and EMA

The attention block at the 16×16 bottleneck improves global object-level reasoning. Since each generated image may contain multiple objects, self-attention helps the model capture relationships between different spatial regions. In addition, EMA stabilizes the sampling result by using a smoothed version of model parameters. Compared with the baseline conditional DDPM, the attention-based model with EMA achieved much higher accuracy.

4. Experimental Results

The final generated images were evaluated using the pretrained ResNet18 evaluator provided by the instructor. The evaluator takes normalized generated images and ground-truth multi-hot labels as input and returns classification accuracy.

The best observed result was obtained with the attention-based conditional DDPM using EMA and classifier-free guidance.

Table 5. Development-stage Comparison of Model Improvements

Version	Main Change	<code>test.json</code>	<code>new_test.json</code>	Observation
Baseline conditional DDPM	Conditional U-Net only	0.1806	0.1310	Weak conditional alignment
+Classifier-free guidance	guidance scale = 2.0	0.6369	0.6429	Large improvement in label consistency
+ Attention + EMA	guidance scale = 2.0	0.7222	0.9048	Better object-level generation
+ Attention + EMA	guidance scale = 2.6	0.8056	0.8452	Both accuracies exceed 0.8

```
=====
test.json Accuracy: 0.8056
=====
Evaluating new_test.json...
Images shape: torch.Size([32, 3, 64, 64])
Labels shape: torch.Size([32, 24])
=====
new_test.json Accuracy: 0.8452
=====
Final Results
test.json accuracy:    0.8056
new_test.json accuracy: 0.8452
```

Fig. 6. Accuracy screenshot of the final generated images evaluated by the pretrained evaluator.

5. Conclusion

In this lab, I first implemented a basic conditional DDPM for multi-label i-CLEVR image generation. I built the data loader to convert object labels into 24-dimensional multi-hot vectors, and used a conditional U-Net as the denoising network. The model was trained with the standard epsilon prediction objective and MSE loss.

After completing the baseline model, I found that the generated images were not well aligned with the given label conditions. Therefore, I added classifier-free guidance by randomly dropping the condition vector during training and using a guidance scale during sampling. This significantly improved the evaluator accuracy, showing that stronger conditional guidance helps the model generate images that better match the target labels.

Next, I further improved the model by adding a self-attention block at the 16×16 bottleneck and using EMA weights during sampling. The attention block helped the model capture relationships between different objects, while EMA made the generated images more stable. I also tested different guidance scales and found that the performance was sensitive to the guidance value.

Finally, the best result was obtained with the attention-based conditional DDPM using EMA and classifier-free guidance. The model achieved 0.8056 accuracy on `test.json` and 0.8452 accuracy on `new_test.json`, both exceeding the 0.8 threshold. Overall, this lab helped me understand the complete conditional DDPM pipeline, including multi-label preprocessing, noise prediction, conditional sampling, and evaluator-based performance analysis.

6. Reference

- [1] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 6840–6851, 2020.

- [2] J. Ho and T. Salimans, “Classifier-free diffusion guidance,” *arXiv preprint arXiv:2207.12598*, 2022. [Online]. Available: <https://arxiv.org/abs/2207.12598>
- [3] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015, pp. 234–241.
- [4] H. Zhang and V. K. U. B. Gupta, “Hugging Face Diffusion Models Course,” Hugging Face, 2023. [Online]. Available: <https://huggingface.co/learn/diffusion-course/en/index>
- [5] A. Nichol and P. Dhariwal, “Improved denoising diffusion probabilistic models,” in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021, pp. 8162–8171.